# ROS2 Spring Workshop

Lab 3: The Invisible Wall (Geofencing & Safety)

**Project Context: The "Wet Zone"**

In **Phase 3** of your Final Project, your robot must avoid a simulated "wet zone" (a virtual mud pit). The robot won't see this with a camera; it must use GPS coordinates to know when to stop. **This lab builds that safety brain.** Today you'll also confront a physics problem that you will *redeem* in Lab 7, pay attention to the Reality Gap at the end.

## Objective

In Lab 2, you built a robot that drove blindly using a timer. In AgTech, a blind robot is a dangerous robot. Today you give the robot **ears** (sensors) so it can listen to its GPS and react to boundaries.

You will learn:

- **Subscribers:** How to read sensor data (Pose/GPS).
- **Callbacks:** How to write code that reacts to events ("interrupts").
- **State Machines:** How to switch behaviours (Drive → Stop → Turn).
- **Safe Shutdown:** Why robots must always stop cleanly when your code exits.

## Pre-Lab Concept Primer

### 1. Open Loop vs. Closed Loop

Real farm robots do not run on timed loops; they react to the world.

- **Open Loop (Lab 2):** "Drive forward for 5 seconds." Dangerous if a ditch appears.
- **Closed Loop (Lab 3):** "Drive forward *until* $X > 9.0$, then stop." Safe.

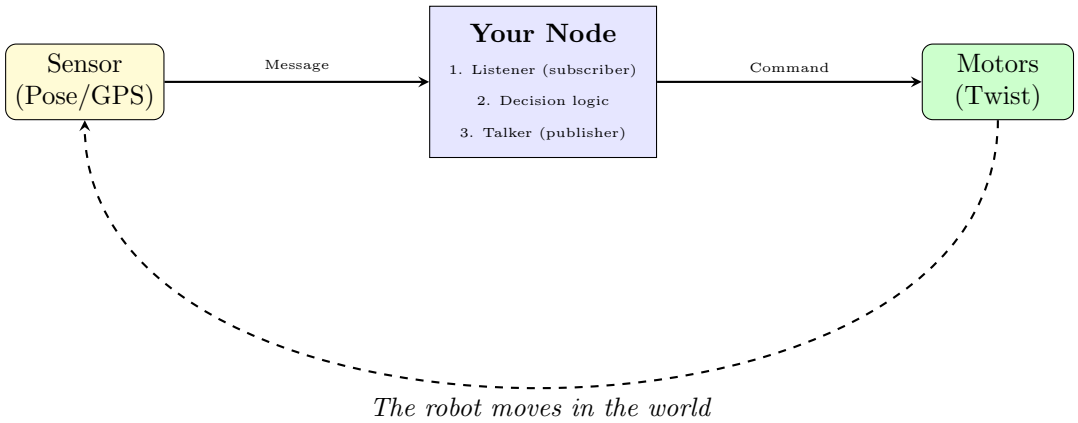


Figure 1: The Feedback Loop: the robot moves, the GPS updates, and the code reacts.

### 2. Timer AND Subscriber (Not Either/Or)

**Important Framing**

In Lab 2 your robot had *only* a timer: it drove blindly. Today you add a subscriber *too*. The timer keeps driving the wheels; the subscriber watches the world. When the subscriber sees something important, it updates a memory flag, and the next timer tick acts on it.
This "timer-for-control + subscriber-for-reaction" pattern is how almost every robot controller is structured. You'll use it in every remaining lab of the semester.

### 3. The "Doorbell" Analogy (The Callback)

Think of your robot's code like a person sitting in a living room.

- **The Main Loop (Timer):** You are busy watching a movie.
- **The Topic:** The sound of the doorbell.
- **The Callback:** The specific action you take *only when the bell rings.*

When the doorbell rings (sensor data arrives), you pause your movie, run to the door, sign for the package, and immediately return to the couch. You do not stand at the door waiting for it to ring; that would be a waste of time.

**For Arduino Users: Is this an ISR?**

Yes! A **Callback** in ROS 2 is exactly like an **Interrupt Service Routine (ISR)**.

- **Arduino ISR:** A hardware pin voltage triggers a function.
- **ROS 2 Callback:** A software message triggers a function.

**CRITICAL WARNING: The "Neighbor" Rule**

**NEVER use `time.sleep()` inside a callback!**
If you open the door and stand there talking to your neighbor for 10 seconds (sleep), you miss the movie! In robotics, if you sleep inside a callback, the robot stops updating its position, stops listening to safety commands, and **will crash** into the fence.

## Part 1: Package Setup

We need to build the container for our code. Even though we're using pre-written logic, we must manually create the ROS 2 package structure to register it with the workspace.

1. **Navigate to your source folder:**

```
cd /workspaces/agtech_ros2/src

```

2. **Create the empty package:**

```
ros2 pkg create --build-type ament_python lab3_safety_brake

```

3. **Verify the structure:** Ensure you see the new folder. This command generated `package.xml` and `setup.py` automatically.

## Part 2: Importing the Logic

Writing a robust safety node takes time. To focus on the *logic* rather than the typing, the source code is provided in your lab resources folder.

**Your task is to install this "brain" into the package you just created.**

1. **Copy the script from resources to your package:**

```
cp /workspaces/agtech_ros2/lab_resources/lab3/safety_stop.py \
   /workspaces/agtech_ros2/src/lab3_safety_brake/lab3_safety_brake/

```

2. **Open the file to verify:**

```
code /workspaces/agtech_ros2/src/lab3_safety_brake/lab3_safety_brake/safety_stop.py

```

**Concept Check: Why did we copy it there?**

Notice the destination path has the package name twice:
`.../lab3_safety_brake/lab3_safety_brake/`
In Python packages, the source code lives inside a subfolder with the same name as the package. If you put the file at the top level (next to `setup.py`), ROS can't find it.

# Part 3: Code Analysis

This is the code you copied in Part 2. Look at the code open in your editor and compare it to the code walkthrough on the next page. This script implements a **geofence**: the robot drives freely, but if it crosses $x = 9.0$, it automatically stops.

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose  # <--- CRITICAL IMPORT!

class SafetyBrake(Node):
    def __init__(self):
        super().__init__('safety_brake_node')

        # 1. THE MOUTH (Publisher)
        self.publisher_ = self.create_publisher(
            Twist, '/turtle1/cmd_vel', 10)

        # 2. THE EARS (Subscriber)
        self.subscription_ = self.create_subscription(
            Pose,
            '/turtle1/pose',
            self.pose_callback,
            10
        )

        # 3. THE HEARTBEAT (Timer)
        self.timer_ = self.create_timer(0.1, self.timer_logic)

        # 4. THE MEMORY (State)
        self.stop_robot = False
        self.get_logger().info("Geofence System Active")

    def timer_logic(self):
        # The Brain: decide WHAT to do based on memory
        msg = Twist()

        if self.stop_robot:
            msg.linear.x = 0.0  # E-BRAKE
            msg.angular.z = 0.0
        else:
            msg.linear.x = 1.0  # Cruise speed
            msg.angular.z = 0.0

        self.publisher_.publish(msg)

    def pose_callback(self, msg):
        # The Reflex: update memory based on senses
        if msg.x > 9.0:
```

```
46              if not self.stop_robot:
47                  self.get_logger().warn(
48                      f"GEOFENCE BREACH AT X={msg.x:.2f}! STOPPING.")
49              self.stop_robot = True
50          else:
51              self.stop_robot = False
52
53
54  def main(args=None):
55      rclpy.init(args=args)
56      node = SafetyBrake()
57      try:
58          rclpy.spin(node)
59      except KeyboardInterrupt:
60          pass
61      finally:
62          # SAFETY: publish zero velocity on shutdown so the robot
63          # doesn't coast after Ctrl+C. You will use this pattern
64          # in every controller you write this semester.
65          stop_msg = Twist()
66          node.publisher_.publish(stop_msg)
67          node.destroy_node()
68          rclpy.shutdown()
69
70
71  if __name__ == '__main__':
72      main()
```

Listing 1: safety_stop.py (for reference only)

## Code Walkthrough: Chunk by Chunk

Let's break down exactly what this script is doing:

- **Lines 1-5 (The Shebang & Imports):** We start with the standard Linux shebang and our usual ROS 2 imports (`rclpy`, `Node`, and `Twist`). However, notice the critical addition on line 5: `from turtlesim.msg import Pose`. We must import this specific message type so our subscriber can understand the spatial coordinate data it is receiving.
- **Lines 7-28 (The Constructor):** The `__init__` method initializes four vital components. First, our Publisher (the "mouth") to send velocity commands. Second, our Subscriber (the "ears") to continuously listen to the `/turtle1/pose` topic. Third, a Timer (the "heartbeat") set to trigger 10 times a second (0.1s). Finally, we create a State or Memory variable (`self.stop_robot = False`) to remember if the geofence has been breached.
- **Lines 30-41 (The Brain):** The `timer_logic` function is driven by our heartbeat timer. Notice that this loop only looks at the memory; it doesn't care about coordinates. If `stop_robot` is `True`, it slams the E-brake (0.0 velocity). If `False`, it cruises forward. By separating the decision-making brain from the sensory reflexes, our code remains highly predictable.
- **Lines 43-51 (The Reflex):** The `pose_callback` function triggers instantly every time the subscriber hears a new location. If the $x$ coordinate crosses 9.0, it trips our memory variable to `True` and logs a warning to your screen.
- **Lines 54-72 (The Engine & Brakes):** The `main()` function keeps the program alive via `rclpy.spin(node)`. Crucially, look at the `finally` block. If you press Ctrl+C, the script dies, but the robot will keep executing its last known velocity command. We explicitly publish a "stop" message right before the node shuts down. When you run this on the physical Ackermann steering robots later, a coasting robot means a crashed robot, so we are building the safety habit here in simulation!

**Professional Habit: Always Stop On Shutdown**

This is the first lab where you'll write a node that commands a robot to move. Notice the `try/except/finally` block in `main()`. When you Ctrl+C, the `finally` block publishes a zero-velocity Twist before the node exits.
**Why it matters:** Without this, the turtle (or a real robot in Lab 8+) would keep driving with its last command until something else stops it. On the physical tractor this is a genuine safety issue. Get in the habit now, because you will reuse this pattern in Labs 6, 7, 8, and 10.

## Part 4: Registration & Build

The code is in the folder, but ROS doesn't know it's an executable program yet.

1. **Edit `setup.py`:** Add the entry point so ROS can find the `main` function.

```
entry_points={
    'console_scripts': [
        'safety_stop = lab3_safety_brake.safety_stop:main',
    ],
},

```

2. **Build:** now would be a great opportunity to try out that `--symlink-install` flag!

```
cd /workspaces/agtech_ros2
colcon build --symlink-install
source install/setup.bash

```

**Linux Tip: What does "source" do?**

**Command:** `source install/setup.bash`
Think of this as refreshing the "phonebook" (environment variables).

- When you build a new package, it sits in the `install` folder, invisible to the system.
- Running `source` tells the terminal: "Look in this folder for new commands."
- **Rule:** You must run this in **every new terminal window**, or ROS won't find your code.

## Part 5: The Test Drive

Now we run the system. We need two terminals: one for the robot (simulator) and one for the brain (your script).

1. **Open Terminal 1 (The Simulator):**

```
ros2 run turtlesim turtlesim_node

```

2. **Open Terminal 2 (The Brain):** *Don't forget to source your workspace!*

```
source install/setup.bash
ros2 run lab3_safety_brake safety_stop

```

**Result:** The turtle should drive to the right and automatically stop when it hits the coordinate $x = 9.0$.

# Part 6: Building the Headland Turn

Stopping at the fence is useful, but a real tractor needs to *turn around* at the end of a row and drive back. This is called a **headland turn**.

We'll build it in two stages so the conceptual leap stays manageable:

- **Stage 1 (warmup):** Refactor your existing geofence into a two-state machine. Same behaviour, new structure. About 5 minutes.
- **Stage 2 (challenge):** Extend the two states into four (FORWARD, TURNING, RETURN, DONE) to produce the full paperclip path.

**AgTech Alert: Tank vs. Tractor Physics**

In **Week 8**, your physical robot uses **Ackermann steering** (like a car). It cannot spin in place.

- **Tank Turn:** `linear.x = 0.0`, `angular.z = 2.0`
  Week 8 robot will stall! This only works in Turtlesim.
- **Tractor Turn:** `linear.x = 0.5`, `angular.z = 1.0`
  Correct arc. Works on both sim and real robot.

For this lab, feel free to use whichever works, you'll confront the real Ackermann constraint in Lab 8.

## Stage 1: From Boolean to State (Warmup)

The geofence you just ran uses a boolean flag (`self.stop_robot`). That works for two behaviours, but it doesn't scale: you can't represent "turning" or "returning" with True/False. Real robot controllers use named states.

Before you add new behaviours, refactor what you already have. The runtime behaviour will be *identical*, but the structure will be ready for Stage 2.

**Three small edits in `safety_stop.py`:**

**1. In `__init__`:** replace the boolean with a state variable.

```
# BEFORE
self.stop_robot = False

# AFTER
self.state = 'FORWARD'   # values: 'FORWARD' or 'STOPPED'
```

**2. In `timer_logic`:** check the state string instead of the boolean.

```
# BEFORE
if self.stop_robot:
    msg.linear.x = 0.0
    msg.angular.z = 0.0
else:
    msg.linear.x = 1.0
    msg.angular.z = 0.0

# AFTER
if self.state == 'STOPPED':
    msg.linear.x = 0.0
    msg.angular.z = 0.0
else:
    msg.linear.x = 1.0
    msg.angular.z = 0.0
```

**3. In `pose_callback`:** update the state string instead of the boolean.

```
# BEFORE
if msg.x > 9.0:
    if not self.stop_robot:
        self.get_logger().warn(
            f"GEOFENCE BREACH AT X={msg.x:.2f}! STOPPING.")
    self.stop_robot = True
else:
    self.stop_robot = False

```

```
# AFTER
if msg.x > 9.0:
    if self.state != 'STOPPED':
        self.get_logger().warn(
            f"GEOFENCE BREACH AT X={msg.x:.2f}! STOPPING.")
    self.state = 'STOPPED'
else:
    self.state = 'FORWARD'
```

**Test it.** Re-run as in Part 5. The turtle should drive to $x = 9.0$ and stop, exactly as before. If it doesn't, the rename is incomplete; check all three spots.

**Why bother with this rename?**

Two reasons:

- **Logs are readable.** `self.get_logger().info(f"state={self.state}")` prints `state=TURNING`, not `state=True`. With four states, this is the difference between debugging in 2 minutes vs. 20.
- **It scales.** Adding a third behaviour with a boolean means adding a second boolean (`self.is_turning`). Then you have to track combinations like "stopped AND not turning". With state strings, you just add another string value.

You've now written your first state machine. Stage 2 just adds two more states.

## Stage 2: Four States, Full Paperclip

**The full state machine:**

1. **FORWARD:** Drive straight east. If $x > 9.0$, switch to TURNING.
2. **TURNING:** Drive AND turn (a tractor arc). When the heading has flipped roughly 180°, switch to RETURN.
3. **RETURN:** Drive straight west.
4. **DONE:** When $x < 1.0$, stop the robot. Mission complete.

### Worked Example: One Transition in Detail

Before you fill in the worksheet, study this single transition. Every other one follows the same pattern.

```
if self.state == 'FORWARD' and msg.x > 9.0:
    self.state = 'TURNING'
    self.get_logger().info("Fence hit. Beginning U-turn.")
```

Read it like English:

- `self.state == 'FORWARD'`: Only fire this transition if we are *currently* driving forward. Without this guard, the line would re-fire every callback after we cross the fence.
- `and msg.x > 9.0`: AND we have crossed the geofence. Both conditions must be true.
- `self.state = 'TURNING'`: Update memory. The next timer tick will see the new state and run the TURNING branch of `timer_logic` instead of the FORWARD branch.

**Every transition has this shape:** check current state, check sensor condition, update state. Now finish the worksheet to write the other two.

### The Worksheet (Required)

**Important: Replace, don't append**

You must **delete the entire body of `pose_callback` from Stage 1** and replace it with the new transition logic below. If you leave both, your old `self.state = 'STOPPED'` line will fight the new `'TURNING'` transition and the robot will never enter the U-turn.
The `timer_logic` function also needs to be expanded to handle four state branches instead of two.

Fill in the blanks. Use string state names (`'FORWARD'`, `'TURNING'`, `'RETURN'`, `'DONE'`) rather than integers.

**Action: what each state tells the motors to do.**

```
def timer_logic(self):
    msg = Twist()

    if self.state == 'FORWARD':
        msg.linear.x  = ________   # Drive fast
        msg.angular.z = 0.0

    elif self.state == 'TURNING':
        msg.linear.x  = 0.5         # Drive slow (tractor arc)
        msg.angular.z = ________    # Turn left

    elif self.state == 'RETURN':
        msg.linear.x  = 1.0
        msg.angular.z = ________    # Drive straight

    elif self.state == 'DONE':
        msg.linear.x  = 0.0         # Stopped
        msg.angular.z = 0.0

    self.publisher_.publish(msg)
```

**Transitions: when to switch states.** The first transition (`FORWARD` → `TURNING`) is filled in for you, it's the worked example from above.

```
def pose_callback(self, msg):
    # 1. Hit the east wall: switch to U-turn
    if self.state == 'FORWARD' and msg.x > 9.0:
        self.state = 'TURNING'
        self.get_logger().info("Fence hit. Beginning U-turn.")

    # 2. Rotated ~180 degrees: switch to return
    # Hint: pi is about 3.14. BUT see the Reality Gap box --
    # this simple check has edge cases we'll fix in Lab 7.
    elif self.state == 'TURNING' and abs(msg.theta) > ________:
        self.state = ________
        self.get_logger().info("Turn complete. Returning.")

    # 3. Crossed back to the west edge: mission complete
    elif self.state == ________ and msg.x < ________:
        self.state = 'DONE'
        self.get_logger().info("Mission complete.")
```

**Suggested values** (tune them later if you want):

| Blank location | Suggested value | What it controls |
|---|---|---|
| `linear.x` forward | 1.0 | Cruise speed |
| `angular.z` turning | 1.0 | Turn rate (tractor arc) |
| `angular.z` return | 0.0 | Straight driving |
| `abs(msg.theta) >` (turn) | 3.0 | "Close enough to $\pi$" |
| `msg.x <` (done) | 1.0 | West edge of mission |

### Test and Observe

Save your changes. Thanks to `--symlink-install` from Part 4, you don't need to rebuild. Just re-run:

```
ros2 run lab3_safety_brake safety_stop
```

You should see the turtle drive east, arc around, drive back west, and stop. The trace should look like a paperclip.

**Look closely at the result, the return path won't be perfectly parallel to the outbound path.** That's not your bug. That's the Reality Gap (next box).

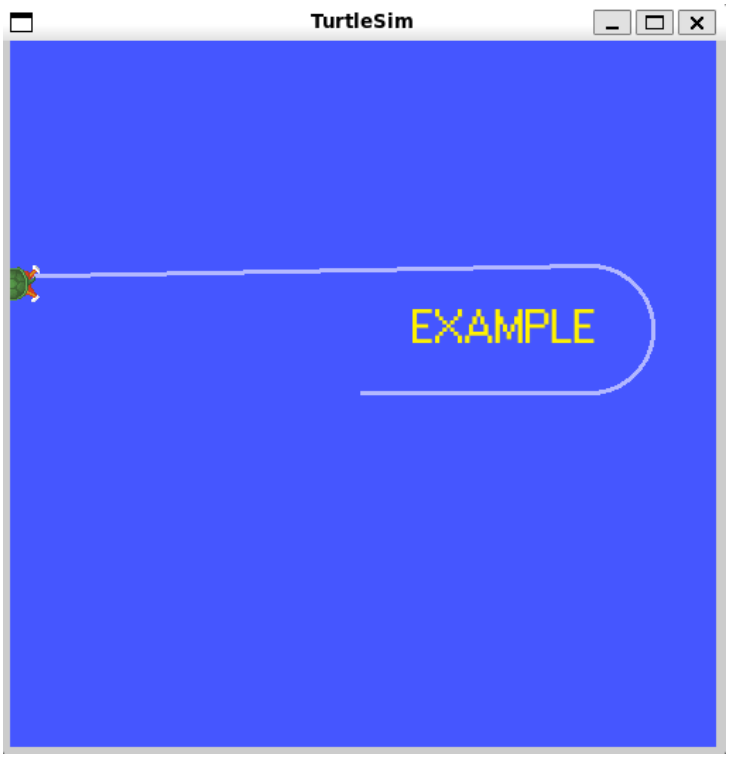


Figure 2: The challenge result: a "paperclip" turn. Note the slight drift on the return path.

**Critical Analysis: The Reality Gap**

Look closely at the result in Figure 2. The return path isn't perfectly parallel: it drifts downwards. Why?
This is not one bug, it's three. Real robotics always has multiple overlapping sources of error:

1. **Sampling rate.** Your timer runs at 10 Hz. Between ticks, the turtle's heading might rotate 15° or more. By the time your code notices "heading $> \pi$", the robot has already overshot.
2. **Angle wraparound.** Turtlesim reports theta in $[-\pi, +\pi]$. When the turtle rotates past $+\pi$, the value snaps to $-\pi$. A simple `theta > 3.14` check gets confused around the wrap point.
3. **No position correction.** During the turn, you aren't checking y-position. The turn happens wherever the turtle drifted *during* the arc, not at a pre-chosen pivot point.

**These are real problems on real robots**, not turtlesim quirks. In Lab 7 you'll fix all three by switching to closed-loop control with breadcrumb waypoints. Remember this feeling, Lab 7 is called "The Redemption Arc" for a reason.

## Part 7: Deliverables

Submit a single PDF containing:

1. **Geofence demo:** A screenshot of turtlesim showing the turtle having stopped near $x = 9.0$, plus the terminal line "GEOFENCE BREACH AT X=... STOPPING." (This works for either Stage 1 or Stage 2; the geofence behaviour is the same.)

2. **Paperclip screenshot:** Your turtle's trace after completing the Stage 2 challenge. The shape should clearly show: forward → U-turn → return.

3. **Code snippet:** The state-transition logic from your final `pose_callback` (the four-state version with FORWARD, TURNING, RETURN, DONE).

4. **Reflection (3 sentences):** From the Reality Gap box, pick *one* of the three bugs causing return-path drift and explain in your own words how it creates the error. Then propose how you'd detect this bug at runtime, for example, what could you log to know when it's happening?

## Troubleshooting

**"My Robot Isn't Updating!"**

**Problem:** You changed the code (e.g., adjusted parameters), but the robot behaves exactly the same as before.
**Cause:** ROS is likely running an old version of your file because the "live link" isn't active or the file wasn't registered.
**Solution:**

1. **The nuclear option:** rebuild with `--symlink-install` to force a live link:

```
cd /workspaces/agtech_ros2
colcon build --symlink-install
source install/setup.bash

```

2. **Check your `setup.py`:** Did you add the new file to the entry points list?
3. **Check your indentation:** Python is very picky. Ensure your methods are indented inside the class.

**"The robot stops at the fence but never turns!"**

**Cause:** You probably left the Stage 1 `pose_callback` body in place. The line `self.state = 'STOPPED'` runs before your new transition logic, so the state never reaches `'TURNING'`.
**Solution:** Open `pose_callback` and confirm the only state assignments inside are to `'TURNING'`, `'RETURN'`, and `'DONE'`. There should be no `'STOPPED'` anywhere in the four-state version.

**"The robot turns forever and never stops!"**

**Cause:** The `TURNING` → `RETURN` condition (`abs(msg.theta) > 3.0`) is never satisfied. Most likely, your turn rate is so slow that the timer-driven robot is wandering around and not actually completing a 180° rotation, OR you have a typo in the threshold.
**Solution:** Add a debug log inside `pose_callback` that prints `msg.theta` every time it fires. Watch the values, do they ever climb past 3.0? If not, increase `angular.z` in the TURNING branch.

**"Both branches fire on the same callback!"**

**Cause:** You used `if` instead of `elif` for the transition checks. With separate `if` statements, multiple transitions can fire on the same pose update.
**Solution:** Use `elif` for transitions 2 and 3 (as shown in the worksheet). Only one branch should be allowed to update the state per callback.

**What's Next: Lab 4**

You've now built a robot that can talk (Lab 2's publisher) and listen (Lab 3's subscriber). That covers the streaming half of ROS: continuous data flowing back and forth.
But what about one-shot transactions? When your tractor needs to plant a seed, take a photo, or reset the simulation, you don't want a stream, you want a request, a confirmation, and a clear "did it work?" answer. That's a different communication pattern entirely.
In Lab 4 you'll meet **Services** (ROS's request/response system) and use it to build a seeder that drops markers exactly where you want them, exactly once.